

**Практические работы №7-8 по дисциплине
«Клиент-серверные приложения»**

Темы: «Знакомство с React» и «React, работа с API»

Выполнил: студент группы БИВТ-24-2

Горбачёв В. А.

2026 г.

1. Введение

В рамках практических работ №7 и №8 был создан учебный frontend-проект на React. Работа выполнялась последовательно: сначала был реализован отдельный компонент визитки студента, затем к проекту была добавлена загрузка данных из API и отображение полученного списка пользователей.

Практическая работа №7 закрепляет базовые понятия React: компонентный подход, JSX-разметку, импорт и подключение компонентов, использование состояния и обработку пользовательского события. Практическая работа №8 показывает следующий уровень работы с React: асинхронный запрос через fetch, применение useEffect для загрузки данных при открытии страницы, хранение результата в useState, отображение состояния загрузки и обработку ошибки.

Итоговый интерфейс объединяет обе работы на одной странице. Слева отображается визитка студента, справа отображается список пользователей, полученный из тестового API. Такой подход удобен для проверки, потому что на одном экране видно выполнение требований обеих практических работ.

2. Цели и задачи работ

2.1. Практическая работа №7

Цель ПР-7 - познакомиться с основами React и реализовать компонент ProfileCard. По заданию компонент должен содержать заголовок «Моя визитка», имя студента, название специальности, номер группы, краткое описание и возможность загрузить изображение для аватарки.

- создать отдельный компонент ProfileCard;
- подключить компонент в App.jsx;
- использовать JSX-разметку;
- использовать несколько HTML-элементов;
- реализовать загрузку изображения для аватарки.

2.2. Практическая работа №8

Цель ПР-8 - реализовать React-компоненты, которые получают данные из API и отображают их пользователю. Компонент должен отправлять запрос при открытии страницы, сохранять полученные данные в состояние, показывать сообщение загрузки, обрабатывать ошибку и выводить список через map().

- выполнить запрос к API при открытии страницы;
- сохранить JSON-данные в состояние компонента;
- вывести данные списком;
- показать «Загрузка...» во время запроса;
- показать «Ошибка загрузки данных» при ошибке;
- использовать `map()` и уникальный `key` для элементов списка.

3. Используемые инструменты

- Visual Studio Code - редактор кода для работы с React-компонентами и стилями.
- Node.js - среда выполнения JavaScript вне браузера.
- npm - менеджер пакетов для установки зависимостей и запуска команд.
- React - библиотека для создания интерфейса из компонентов.
- Vite - инструмент разработки и сборки React-проекта.
- Браузер - проверка результата работы приложения и просмотр JSON-данных API.
- PowerShell - командная строка для выполнения `npm.cmd install` и `npm.cmd run build`.

4. Теория React

React - это JavaScript-библиотека для создания пользовательских интерфейсов. Основная идея React заключается в разбиении интерфейса на компоненты. Компонент описывает самостоятельную часть страницы и может быть использован внутри других компонентов.

В данной работе используются функциональные компоненты. Такой компонент является функцией JavaScript, которая возвращает JSX-разметку. JSX похож на HTML, но находится внутри JavaScript-кода. Поэтому в JSX используются `className` вместо `class`, фигурные скобки для вставки значений и `camelCase`-атрибуты для событий.

`useState` - это хук React для хранения изменяемых данных внутри компонента. В `ProfileCard` состояние `avatar` хранит выбранное изображение. В `UsersList` состояние `users` хранит массив пользователей, `loading` хранит признак загрузки, `error` хранит текст ошибки.

`useEffect` - это хук для выполнения побочных действий. К таким действиям относится запрос к API. В `UsersList` `useEffect` запускает загрузку пользователей один раз после первой отрисовки компонента, потому что вторым аргументом передан пустой массив зависимостей `[]`.

5. Выполнение практической работы №7

Для ПР-7 был создан компонент ProfileCard. Он отвечает за отображение визитки студента. Внутри компонента размещены заголовок, блок аватарки, кнопка выбора изображения, ФИО студента, специальность, группа и список кратких пунктов с описанием.

Загрузка аватарки реализована через `input type="file"`. Пользователь выбирает файл, после чего обработчик `handleAvatarChange` получает первый выбранный файл из `event.target.files`. Если файл не выбран, функция завершает работу. Если файл есть, создается временный URL изображения с помощью `URL.createObjectURL(file)`, и этот адрес записывается в состояние `avatar`.

В JSX используется условный рендеринг: если `avatar` содержит адрес изображения, отображается тег `img`. Если `avatar` пустой, вместо изображения выводятся инициалы студента. Такой подход делает компонент удобным: он корректно выглядит до выбора файла и после выбора файла.

Фрагмент ProfileCard

```
function ProfileCard() {
  const [avatar, setAvatar] = useState("");

  const handleAvatarChange = (event) => {
    const file = event.target.files[0];
    if (!file) return;
    setAvatar(URL.createObjectURL(file));
  };

  return (
    <article className="profile-card">
      <h1>Моя визитка</h1>
      {avatar ? <img src={avatar} alt="Аватар студента" /> : <span>ВГ</span>}
    </article>
  );
}
```

6. Теория API и JSON

API - это интерфейс, через который клиентское приложение может получать данные от сервера. В веб-приложениях API часто возвращает данные в формате JSON. JSON удобен для JavaScript, потому что после преобразования `response.json()` данные становятся обычными объектами и массивами.

В React запросы к API обычно выполняются внутри `useEffect`. Это необходимо, чтобы запрос не запускался при каждой перерисовке компонента. Если запрос написать прямо в теле функции компонента, обновление состояния может приводить к повторному запросу и бесконечному циклу обновлений.

Функция `fetch` выполняет HTTP-запрос и возвращает `Promise`. В работе используется синтаксис `async/await`. Такой вариант удобен для чтения: сначала ожидается ответ сервера, затем проверяется `response.ok`, потом вызывается `response.json()`, после чего данные сохраняются через `setUsers`.

При работе с API важно учитывать промежуточные состояния. Пока данные не пришли, пользователь должен видеть сообщение загрузки. Если произошла ошибка сети, неверный URL или сервер вернул ошибочный статус, интерфейс должен показать понятное сообщение об ошибке.

7. Выполнение практической работы №8

Для ПР-8 создан компонент `UsersList`. Он выполняет запрос к локальному тестовому API `/api/users.json`. Этот файл расположен в папке `public/api`, поэтому при запуске Vite он доступен в браузере как обычный HTTP-ресурс.

В компоненте объявлены три состояния: `users`, `loading` и `error`. `users` хранит массив пользователей. `loading` показывает, выполняется ли запрос. `error` хранит текст ошибки, если запрос завершился неудачно.

Внутри `useEffect` объявлена асинхронная функция `loadUsers`. Она устанавливает `loading` в `true`, очищает ошибку, выполняет `fetch`-запрос, проверяет `response.ok`, преобразует ответ в JSON и сохраняет массив в `users`. В `catch` записывается сообщение «Ошибка загрузки данных». В `finally` загрузка выключается через `setLoading(false)`.

После успешной загрузки данные выводятся списком. Для обхода массива используется `users.map()`. Каждый пользователь передается в компонент `UserCard`. У компонента указан `key={user.id}`, потому что React требует уникальный ключ для элементов списка.

Фрагмент `UsersList`

```
const [users, setUsers] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState("");

useEffect(() => {
  const loadUsers = async () => {
    try {
      setLoading(true);
      setError("");
      const response = await fetch("/api/users.json");
      if (!response.ok) throw new Error("Ошибка ответа сервера");
      const data = await response.json();
      setUsers(data);
    } catch {
      setError("Ошибка загрузки данных");
    } finally {

```

```
        setLoading(false);
    }
};
loadUsers();
}, []);
```

Вывод списка через map()

```
<ul className="data-list">
  {users.map((user) => (
    <UserCard key={user.id} user={user} />
  ))}
</ul>
```

8. Структура проекта

Проект ПР-8 находится в отдельной папке лр8. Он содержит исходные файлы React-приложения, тестовый JSON API, стили, зависимости и скриншоты для отчета.

```
лр8/
├── index.html
├── package.json
├── package-lock.json
├── README.md
├── public/
│   └── api/
│       └── users.json
├── src/
│   ├── main.jsx
│   ├── App.jsx
│   ├── styles.css
│   └── components/
│       ├── ProfileCard.jsx
│       ├── UsersList.jsx
│       └── UserCard.jsx
└── screens/
    ├── 1.png
    ├── 2.png
    ├── 3.png
    ├── 4.png
    ├── 5.png
    └── 6.png
```

Файл package.json содержит команды dev, build и preview. Команда dev используется для режима разработки, build - для проверки и production-сборки, preview - для просмотра собранного проекта.

```
"scripts": {
  "dev": "vite --host 127.0.0.1",
  "build": "vite build",
  "preview": "vite preview --host 127.0.0.1"
}
```

9. Подробный разбор компонентов

9.1. App.jsx

App.jsx является главным компонентом. Он импортирует ProfileCard и UsersList, а затем размещает их в общей секции work-area. Вся логика вынесена в дочерние компоненты, поэтому App.jsx остается коротким и отвечает только за композицию интерфейса.

```
function App() {  
  return (  
    <main className="app">  
      <section className="work-area">  
        <ProfileCard />  
        <UsersList />  
      </section>  
    </main>  
  );  
}
```

9.2. ProfileCard.jsx

ProfileCard является компонентом ПР-7. Он демонстрирует локальное состояние, обработку события выбора файла и условное отображение аватарки. Компонент не обращается к серверу, потому что его задача - показать работу с JSX и пользовательским вводом.

9.3. UsersList.jsx

UsersList является компонентом ПР-8. Он отвечает за загрузку данных и выбор состояния интерфейса: загрузка, ошибка или готовый список. Такой компонент можно назвать контейнерным, потому что он управляет данными и передает их дальше.

9.4. UserCard.jsx

UserCard является презентационным компонентом. Он получает объект пользователя через props и отображает поля name, email и address.city. Вынесение одного элемента списка в отдельный компонент делает структуру проекта более аккуратной.

9.5. users.json

users.json имитирует ответ API. Каждый объект имеет уникальный id, имя пользователя, email и город. Наличие id важно, потому что именно это поле используется как key при выводе массива через map().

```
{  
  "id": 1,  
  "name": "Leanne Graham",  
  "email": "Sincere@april.biz",
```

```
"address": {  
  "city": "Gwenborough"  
}  
}
```

10. Проверка соответствия требованиям задания

10.1. Требования ПР-7

- Создан отдельный компонент ProfileCard - выполнено.
- Компонент подключен в App.jsx - выполнено.
- Есть заголовок «Моя визитка» - выполнено.
- Есть имя студента - выполнено.
- Есть специальность и группа - выполнено.
- Есть краткое описание - выполнено.
- Есть загрузка изображения для аватарки - выполнено.
- Используется JSX и больше трех HTML-элементов - выполнено.

10.2. Требования ПР-8

- Реализовано несколько React-компонентов - выполнено.
- При открытии страницы выполняется запрос к API - выполнено.
- Полученные данные сохраняются в состояние users - выполнено.
- Данные выводятся списком - выполнено.
- Во время загрузки отображается «Загрузка...» - выполнено.
- При ошибке отображается «Ошибка загрузки данных» - выполнено.
- Для вывода списка используется map() - выполнено.
- У каждого элемента списка есть уникальный key={user.id} - выполнено.

11. Проверка запуска и сборки

Перед запуском проекта необходимо перейти в папку лр8. Это важно, потому что команды npm выполняются относительно текущей папки. Если запустить npm.cmd run build из корня, будет запущен другой package.json.

```
cd .\лр8  
npm.cmd install  
npm.cmd run build
```


Сборка была выполнена успешно. Vite обработал React-компоненты, CSS-стили и JSON-данные, после чего сформировал папку dist. Успешная сборка подтверждает отсутствие синтаксических ошибок и корректность импортов.

12. Скриншоты выполнения работы

12.1. Рисунок 1 - общий вид приложения: визитка студента и пользователи из API

Скриншот показывает итоговый интерфейс, где объединены результаты двух практических работ. Левая часть относится к ПР-7 и демонстрирует компонент визитки студента. Правая часть относится к ПР-8 и показывает список пользователей, полученный через API-запрос.

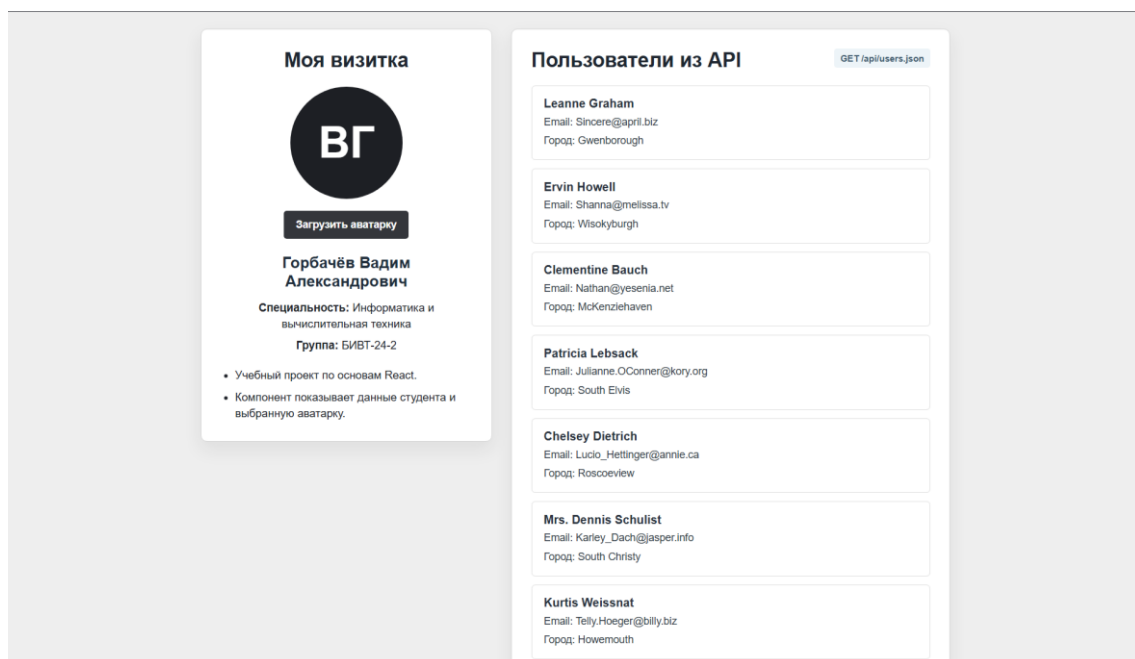


Рисунок 1 - общий вид приложения: визитка студента и пользователи из API

12.2. Рисунок 2 - компонент UsersList с загрузкой данных

На скриншоте показан компонент, который выполняет запрос к API. Видны состояния `users`, `loading` и `error`, хук `useEffect`, функция `fetch`, преобразование ответа в JSON и вывод массива через `map()`.

```

ap8 > src > components > UsersList.jsx > ...
1  import React, { useEffect, useState } from "react";
2  import UserCard from "../UserCard.jsx";
3
4  function UsersList() {
5      const [users, setUsers] = useState([]);
6      const [loading, setLoading] = useState(true);
7      const [error, setError] = useState("");
8
9      useEffect(() => {
10         const loadUsers = async () => {
11             try {
12                 setLoading(true);
13                 setError("");
14
15                 await new Promise((resolve) => setTimeout(resolve, 700));
16
17                 const response = await fetch("/api/users.json");
18
19                 if (!response.ok) {
20                     throw new Error("Ошибка ответа сервера");
21                 }
22
23                 const data = await response.json();
24                 setUsers(data);
25             } catch {
26                 setError("Ошибка загрузки данных");
27             } finally {
28                 setLoading(false);
29             }
30         };
31
32         loadUsers();
33     }, []);
34
35     return (
36         <article className="data-panel">
37             <div className="panel-title">
38                 <h2>Пользователи из API</h2>
39                 <span>GET /api/users.json</span>
40             </div>
41
42             {loading && <p className="state-message">Загрузка...</p>}
43             {error && <p className="state-message error">{error}</p>}
44
45             {!loading && !error && (
46                 <ul className="data-list">
47                     {users.map((user) => (
48                         <UserCard key={user.id} user={user} />
49                     ))}
50                 </ul>
51             )}
52         </article>
53     );
54 }
55
56 export default UsersList;

```

Рисунок 2 - компонент UsersList с загрузкой данных

12.3. Рисунок 3 - компонент UserCard

Компонент UserCard отвечает за вывод одного пользователя. Он получает данные через props и отображает имя, email и город. Разделение на UsersList и UserCard делает код более понятным.

```
лр8 > src > components > UserCard.jsx > ...
1  import React from "react";
2
3  function UserCard({ user }) {
4    return (
5      <li className="data-item">
6        <strong>{user.name}</strong>
7        <span>Email: {user.email}</span>
8        <span>Город: {user.address.city}</span>
9      </li>
10   );
11 }
12
13 export default UserCard;
14
```

Рисунок 3 - компонент UserCard

12.4. Рисунок 4 - компонент ProfileCard

Компонент ProfileCard реализует визитку студента: заголовок, данные студента, описание и загрузку аватарки. Это соответствует требованиям практической работы №7.

```
2
3  function ProfileCard() {
4      const [avatar, setAvatar] = useState("");
5
6      const handleAvatarChange = (event) => {
7          const file = event.target.files[0];
8
9          if (!file) {
10             return;
11         }
12
13         setAvatar(URL.createObjectURL(file));
14     };
15
16     useEffect(() => {
17         return () => {
18             if (avatar) {
19                 URL.revokeObjectURL(avatar);
20             }
21         };
22     }, [avatar]);
23
24     return (
25         <article className="profile-card">
26             <h1>Моя визитка</h1>
27
28             <div className="avatar-preview">
29                 {avatar ? <img src={avatar} alt="Аватар студента" /> : <span>ВГ</span>}
30             </div>
31
32             <label className="avatar-upload">
33                 Загрузить аватарку
34                 <input type="file" accept="image/*" onChange={handleAvatarChange} />
35             </label>
36
37             <h2>Горбачёв Вадим Александрович</h2>
38
39             <p>
40                 <strong>Специальность:</strong> Информатика и вычислительная техника
41             </p>
42             <p>
43                 <strong>Группа:</strong> БИВТ-24-2
44             </p>
45
46             <ul className="profile-notes">
47                 <li>Учебный проект по основам React.</li>
48                 <li>Компонент показывает данные студента и выбранную аватарку.</li>
49             </ul>
50         </article>
51     );
```

Рисунок 4 - компонент ProfileCard

12.5. Рисунок 5 - JSON-данные users.json

Файл `users.json` используется как тестовый API. React-приложение обращается к нему по адресу `/api/users.json` и получает массив объектов пользователей.

```
[
  {
    "id": 1,
    "name": "Leanne Graham",
    "email": "Sincere@april.biz",
    "address": {
      "city": "Gwenborough"
    }
  },
  {
    "id": 2,
    "name": "Ervin Howell",
    "email": "Shanna@melissa.tv",
    "address": {
      "city": "Wisokyburgh"
    }
  },
  {
    "id": 3,
    "name": "Clementine Bauch",
    "email": "Nathan@yesenia.net",
    "address": {
      "city": "McKenziehaven"
    }
  },
  {
    "id": 4,
    "name": "Patricia Lebsack",
    "email": "Julianne.OConner@kory.org",
    "address": {
      "city": "South Elvis"
    }
  },
  {
    "id": 5,
    "name": "Chelsey Dietrich",
    "email": "Lucio_Hettinger@annie.ca",
    "address": {
      "city": "Roscoeview"
    }
  },
  {
    "id": 6,
    "name": "Mrs. Dennis Schulist",
    "email": "Karley_Dach@jasper.info",
    "address": {
      "city": "South Christy"
    }
  }
]
```

Рисунок 5 - JSON-данные users.json

12.6. Рисунок 6 - успешная сборка проекта

Команда `npm.cmd run build` выполнена успешно. Это подтверждает, что проект корректно собирается Vite и готов к запуску в production-режиме.

```
C:\Users\Vadim\PycharmProjects\клиент-серверные приложения\лр1\лр8>npm.cmd run build

> pr8-react-api@1.0.0 build
> vite build

vite v7.3.3 building client environment for production...
✓ 27 modules transformed.
dist/index.html          0.40 kB | gzip: 0.28 kB
dist/assets/index-Dfsfx_PA.css  2.21 kB | gzip: 0.88 kB
dist/assets/index-B0-zA_9T.js 195.25 kB | gzip: 61.62 kB
✓ built in 986ms
```

Рисунок 6 - успешная сборка проекта

13. Заключение

В результате выполнения практической работы №7 был создан компонент `ProfileCard`, который демонстрирует базовые возможности React: JSX-разметку, состояние компонента, обработку события выбора файла и условное отображение аватарки.

В результате выполнения практической работы №8 был создан компонент `UsersList`, который получает данные из API, сохраняет их в состояние, показывает загрузку, обрабатывает ошибку и выводит список пользователей через `map()`. Для отображения одного пользователя создан компонент `UserCard`.

Итоговый проект показывает типичный принцип построения React-приложения: главный компонент собирает интерфейс, дочерние компоненты отвечают за отдельные части страницы, данные хранятся в состоянии, а внешние данные загружаются через API.

Все обязательные требования практических работ №7 и №8 выполнены. Проект успешно собирается командой `npm.cmd run build`, что подтверждает корректность структуры, импортов и JSX-кода.